Centralized Scheduling in Satellite Computing

An LP-based Approach

Bachelor's Degree in Computer Science

Advisor: Prof. Emiliano Casalicchio

Claudio Bagini (2045337)

Academic Year 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA



Orbital Edge Computing

From passive relays to computing in orbit

- **Bent-pipe model**: satellites forward raw data to ground stations ⇒ downlink bottleneck, long delays
- **Orbital Edge Computing (OEC)**: onboard processors compute *in space*; only results are downlinked
- **LEO megaconstellations** (Starlink, OneWeb, Kuiper)
  - 300–2000 km altitude, ≈7.5 km/s ⇒ continuously changing topology
  - Inter-Satellite Links (ISLs): optical or RF
- Target applications: disaster response, maritime navigation, defense surveillance ⇒ sub-second deadlines



Problem and research question

Task offloading and routing in LEO networks

OEC constraints

- Limited battery / solar energy
- Constrained onboard hardware
- Dynamic topology, intermittent links
- Strict deadlines:
  $T_{up} + T_{cpu} + T_{down} \leq D_r$

ILP trade-off

- Globally optimal, but NP-hard
- One ILP per task: too costly
- Batching: better solutions, but waiting time eats deadlines

Research question

To what extent can a centralized optimization approach maintain global allocation efficiency in high-density satellite networks before queuing and solver overhead cause systemic performance degradation?



Objective and contributions

What this thesis delivers

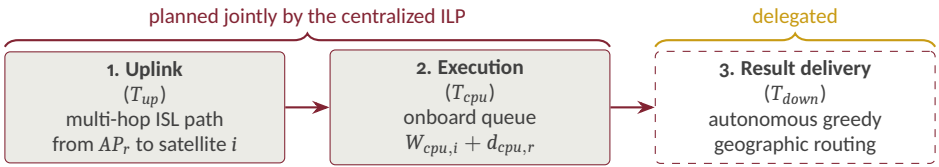
1. **ILP model** for joint ISL routing and CPU allocation, switchable between *Mapping Time* and *Mapping Energy*
2. **Dynamic batching** with Batch Size BS and Batch Timeout BT, plus deadline compensation for queuing delay
3. **Centralized orchestrator** in Python: SimPy (discrete-event), PuLP, HiGHS
4. **Stress and bottleneck analysis**: arrival rates up to $\lambda = 10$ req/s
5. **Benchmark** vs OrbitAware, DTS-base, DTS-Optimal and distributed ILP

System model

Regional Walker-Delta constellation with a Master Orchestrator

Request $r = (d_{cpu,r},\ D_r,\ s_r,\ t_{arr},\ AP_r)$

Master has global state: queues, batteries, ISLs



Dynamic batching engine

Event-driven, controlled by BS and BT



- BS caps $|R|$, hence the $|R| \times |S|$ binary variables: bounded solve time
- BT caps the waiting time of any request: no starvation at low traffic
- Zero-delay barrier `yield Timeout(env,0)` commits all simultaneous events before reading telemetry (state consistency)
- Transient failures (no route, infeasible) are re-enqueued keeping the original t_{arr}



Centralized ILP formulation

Per batch R, over candidate satellites S

Variables

$x_{r,i} \in \{0,1\}$ assign r to i; $y_r \in \{0,1\}$ reject r

Objective (Time-first)

$$\min \ \alpha Z_{time} + \beta Z_{energy} + \gamma Z_{sunset}$$

$$Z_{time} = \sum_{r,i} x_{r,i} R_{r,i} + P_{drop} \sum_{r} y_r$$

$$Z_{energy} = \sum_{r,i} x_{r,i} E_{r,i} + P_{drop} \sum_{r} y_r$$

- $R_{r,i}$: uplink latency + queues + CPU exec
- $E_{r,i}$: ISL forwarding (E^{up}) + CPU energy (E^{cpu})
- $Z_{sunset} = \sum_{r,i} x_{r,i} \left(1 - \frac{t_{sunset,i}}{t_{max}}\right)$
- Energy-first swaps $\alpha \leftrightarrow \beta$

Constraints

$\sum_i x_{r,i} + y_r = 1$	(uniqueness)
$x_{r,i} R_{r,i} \le D_r$	(deadline)
$\sum_{r,i} x_{r,i} \epsilon_{r,i,k} \le B_k$	(battery)
$\sum_r x_{r,i} \le \lfloor \lvert R \rvert / \lvert S \rvert \rfloor + 5$	(load)

Rejected tasks get a cascade diagnosis: no route, deadline, sunset, energy, capacity or global conflict.

Experimental setup

SimPy discrete-event simulation

Environment

- Walker-Delta LEO, 600 km, 53°, optical ISLs, SGP4 orbits
- Arrival rate $\lambda \in \{2, 3, 4, 6, 8, 10\}$ req/s
- Battery per pass: 40 kJ or 80 kJ
- Payloads 1–50 MB

Workload profiles β

- Mixed $(0.40, 0.35, 0.25)$
- CPU-intensive $(0, 1, 0)$
- Data-intensive $(0, 0, 1)$

9 algorithms

- *Distributed heuristics*: DTS-base, DTS-Optimal, OrbitAware
- *Hierarchical ILP*: Time, Energy
- *Centralized ILP (ours)*: fast-dispatch (BS=20, BT=0.05 s) and small-batching (BS=2, BT=2 s), each Time/Energy

Metrics

- Success rate
- Response time: $T_{exec} + T_{net+wait}$
- Rejection causes

Two-tier parameter calibration

Macro (batching) first, then micro (objective weights)

1. Macro: $BS \times BT$ (28 configs)

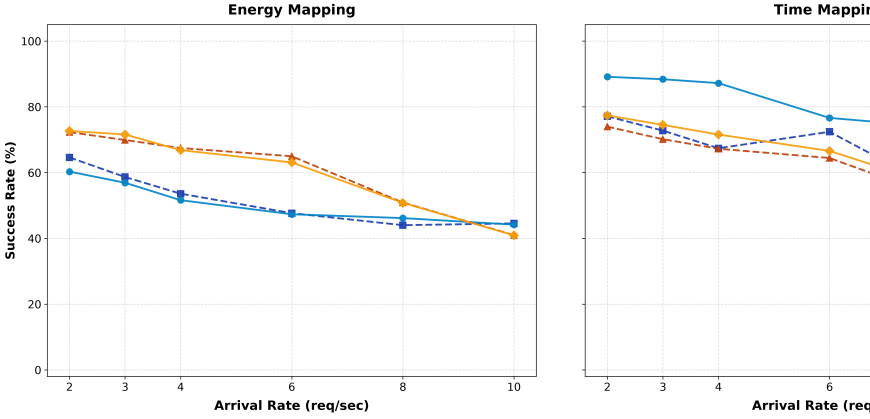
- $BT \geq 0.2$ s or $BS \geq 6$: unviable, 70–85% deadline drops
- $BT = 0$ (single task): myopic greedy, only $\approx$41–48% at $\lambda = 10$
- Two viable archetypes: **fast-dispatch** (20, 0.05 s) and **small-batching** (2, 2 s)

2. Micro: $\gamma \times \alpha$

- Selected $\alpha^* = 1000$, $\gamma^* = 10$
- $\gamma \geq 1000$: *path bloat*, response time $+55\%$ (circuitous detours)
- $\gamma = 10$: eclipse drops $= 0\%$

Calibration result: pre- vs post-optimization

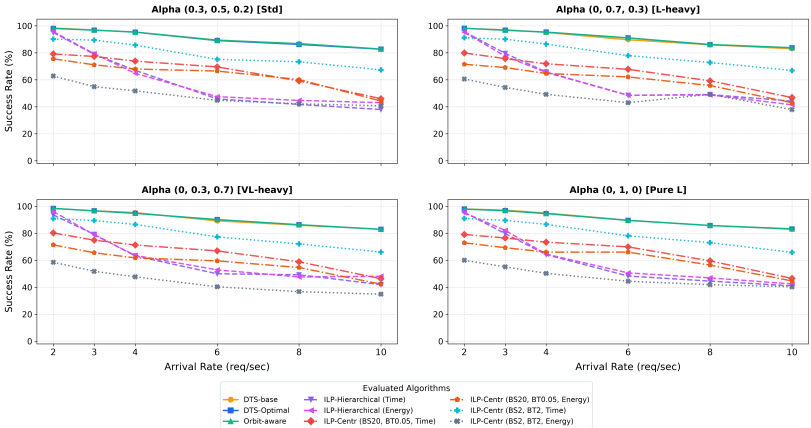
Success rate, Dijkstra-Time routing, $\Delta D = 10\%$



Time Mapping, BS**=2:** 77%→89% (λ=2), 67.5%→87% (4), 54%→73.5% (8), 51%→67% (10): up to +19.5%. Energy Mapping and fast-dispatch are almost insensitive to the weights.

Benchmark: mixed workload

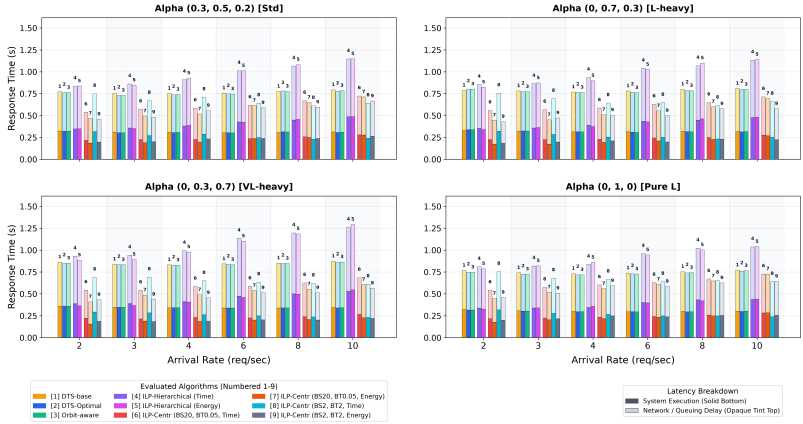
Success rate, 80 kJ, standard payload profile



- **Heuristics**: 98–99% → 79–83% (no ingress queue)
- **ILP-Centr (2, 2 s, Time)**: 90–92% → 65–67%, best centralized; +12–20% over fast-dispatch at $\lambda = 10$
- **Hierarchical ILP**: 97–99% → 39–45%, coordination delays
- **Small-batching latency shrinks with load**: 0.72–0.76 s → 0.56–0.65 s

Response time: computation vs waiting

Breakdown into Onboard Execution (T_{exec}) and Network/Wait Delay ($T_{net+wait}$)



Onboard execution (T_{exec})

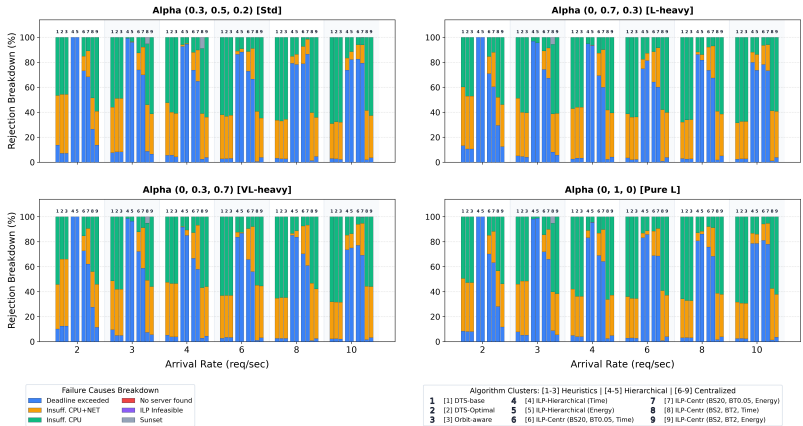
- **Invariant and bounded**: 0.17–0.48 s across all algorithms and loads.
- Single-satellite CPU is never the bottleneck.

Network and queuing delay ($T_{net+wait}$)

- **Heuristics**: flat latency (0.70–0.80 s), zero ingress buffer.
- **Hierarchical ILP**: rapid inflation up to 1.28 s due to inter-cluster coordination delays.
- **Small-batching (2, 2 s, Time)**: *latency compression* as load grows (0.76 s $\rightarrow$ 0.56 s), since bursts saturate $BS = 2$ instantaneously.

Failure diagnostics: what causes task drops?

Physical resource saturation vs deadline expiration



Divergence across workload profiles

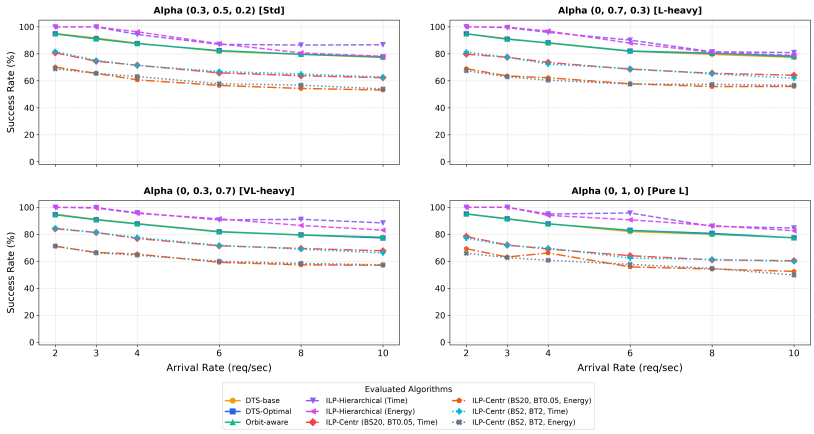
- **CPU-intensive**: 0% link saturation. Heuristics drop 100% due to *Insufficient CPU*.
- **Data-intensive**: 0% CPU drops. Over 95–100% of drops are due to *Insufficient CPU+NET* (laser bandwidth and transponder energy).

The batching-deadline dilemma

- **Fast-dispatch / Hierarchical**: under stress, suffer 65–85% drops from Deadline Exceeded (ingress queues erode slack).
- **Small-batching (2, 2 s)**: deadline drops plunge to $< 2\%$ (mixed) and 8–12% (CPU).

Benchmark: the ranking depends on the workload

Data-intensive, 80 kJ

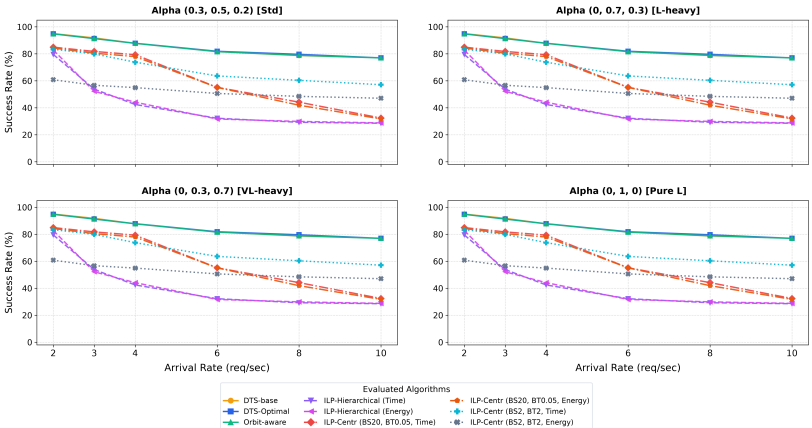


Data-intensive $(0, 0, 1)$

- **Hierarchical ILP (Time) wins**: 99–100% → 81–89% vs heuristics 94–95% → 77–78%
- **Bottleneck**: links and transceiver energy (>95% of drops)
- **80 kJ vs 40 kJ**: up to +9–15%

Benchmark: CPU-intensive workload

Success rate, 80 kJ, $\beta = (0, 1, 0)$



- **Payload profile has *no effect***: tasks carry negligible data
- **Heuristics**: 94–95% → 76–77%
- **Fast-dispatch**: 84–85% → 31–33%, deadline drops up to 75–78%
- **Small-batching (Time)**: 56–58% at $\lambda = 10$, up to +25% over fast-dispatch
- **Hierarchical ILP**: 80–84% → 28–30%



Complexity and scalability

Asymptotic analysis

- **Topology mapping**: multi-source Dijkstra, $O(|AP| \cdot |S| \log |S|)$, since node degree ≤ 4 gives $|E| = O(|S|)$
- **ILP size**: $|R| \cdot |S| + |R|$ binaries; with $BS \leq 20$, $|S| \approx 15$: at most 300 binaries and 330 rows
- **Solver time**: ≈0.30–0.60 ms, independent of the arrival rate λ
- **Forwarding**: planned path $O(h_{up})$; greedy fallback $O(k \log k) \approx O(1)$ with $k \leq 4$
- **Telemetry**: $O(|S|)$ messages per update; $< 0.1\%$ of ISL bandwidth for $|S| \leq 50$, but a limit for thousands of satellites



Deployment guidelines

Which paradigm, for which mission

Compute-Intensive $\beta = (0, 1, 0)$

Recommended: **Distributed Heuristics**

- Zero coordinator queuing
- Prevents deadline collapse
- **72–95%** success, 100% CPU-bounded drops

Data-Intensive $\beta = (0, 0, 1)$

Recommended: **Hierarchical ILP (Time)**

- Resolves ISL transceiver bottlenecks
- High battery elasticity (+9–15% with 80 kJ)
- **81–89%** success (beats heuristics)

Mixed Workload — $\lambda \leq 4$ req/s

Recommended: **Centralized Small-Batching**

- Pairwise ILP ($BS = 2$, $BT = 2$ s, Time)
- Latency compresses: $0.76 \rightarrow 0.56$ s
- High completion: **82–92%** success

Mixed Workload — $\lambda \geq 6$ req/s

Recommended: **Distributed Heuristics**

- Raw admission priority
- Immune to ingress buffer saturation
- Highest sustained yield: **79–83%**

Key insight: Orchestration optimality is fundamentally workload-dimensional.

Conclusions

Answering the research question

The ceiling of centralized orchestration is architectural, not computational

- With $BS \leq 20$, HiGHS solves in ≈0.30–0.60 ms
- Onboard execution is bounded: $T_{exec} \approx 0.17$–0.48 s
- Collapse above $\lambda \geq 6$ is driven by **ingress queuing**, which consumes the deadline slack
- Best centralized: small-batching (BS=2, BT=2 s) with $\alpha = 1000$, $\gamma = 10$: 54–67% at $\lambda = 10$ (mixed)

Limitations

- Batching dilemma for sub-second services
- Master node is a single point of failure (radiation, reboots, maneuvers)
- Telemetry overhead and stale state at LEO speeds

Future work

Research directions

- **Two-tier hybrid orchestration**
  Heuristic fast path for compute-intensive tasks, ILP layer for data-intensive ones
- **Adaptive batching with reinforcement learning**
  Tune $BT(t)$ from traffic rate, residual slack and CPU queues
- **Multi-agent distributed RL (MADRL)**
  Remove the single point of failure; one-hop observations, CTDE training
- **Hardware-in-the-loop prototyping**
  HiGHS on space-grade SoCs (ARM Cortex-R52/A53, NOEL-V)



Centralized Scheduling in Satellite Computing

Thank you for listening!

Any questions?